

On the main landing page of local applications, JVisualVM lists the host name, operating system and the memory information—including the physical and swap size.



Notes: 1. If there is just a progress bar on the bottom that says it is opening the process but it does not open for a long time, go to Tools --> Options --> Network and choose the *No Proxy* option. After this, restart *jvisualvm*. This step may have to be repeated each time the *hsperfdata* directory is removed and re-created.

2. On the main page, JVisualVM shows the overview information. This contains the process ID, main class, arguments to the JVM and the program.

3. The *system properties* available to this JVM can be seen by clicking the *System properties* tab

Monitor

The *Monitor* tab gives an overview about the CPU, memory, threads and the classes. The *CPU* tab gives information on the CPU utilisation by this Java process and the percentage of time spent in garbage collection.

The *Classes* tab gives information on the classes loaded and unloaded. In JVMs where class sharing is enabled, the *Classes* tab shows information on the shared classes. This tab also shows the number of classes loaded and the number of classes unloaded, both for JVM specific and shared classes.

The *Threads* tab gives information on the number of live threads and daemon threads.

Live threads show the number of threads currently started. The *Live peak* shows the highest number of simultaneous live threads that were running in the JVM since it was started. *Total started* gives the total number of threads started so far in the JVM. So, while *live* and *daemon* threads give the current information, *Live peak* and *Total started* take into account the historical (accumulated) information as well.

The memory section gives information on the memory utilisation in heap and the PermGen areas. The information consists of the current size, maximum size and the amount used in the corresponding memory area.

The heap usage graph can be used to quickly identify a memory leak. This graph typically resembles a ‘saw-tooth’. The memory utilisation increases to a threshold level and falls back to a base level after a garbage collection cycle. In programs that have memory leaks, the troughs in the graph after a GC cycle show an increasing trend, i.e., a line drawn connecting the troughs in this graph will have an increasing slope.

PermGen usage also can be viewed from the *PermGen* tab. In most programs, this graph would be horizontal, indicating a constant usage of PermGen. Prior to Java 7, a runtime constant pool was allocated from the PermGen and, hence, if the number of constants created in the program was huge, there was a risk of running out of PermGen space. Nevertheless, it is helpful to observe both the heap and PermGen spaces on an Oracle JVM

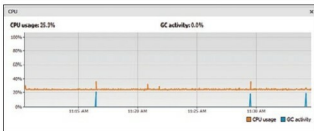


Figure 1: JVisualVM CPU usage

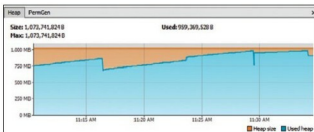


Figure 2: JVisualVM heap usage



Figure 3: JVisualVM threads timeline view

while troubleshooting memory issues.

The two buttons on top of the *Memory* tab can be used to force a garbage collection or take a heap dump. The *Heap Dump* button can be used to manually trigger a heap dump that can be loaded at a later point of time, for examination.

Threads

The *Threads* tab gives the complete details of all the threads—both live and the completed threads. *Timeline view* shows a timeline of the threads that can be scrolled to view the state changes that a thread underwent ever since it was started, right up to the time it terminated. Each state of the thread is indicated by a different colour.